Implementação e Validação em Hardware de um Processador MIPS Monociclo com Suporte a Instruções de Salto e Execução do Algoritmo Sort

José Dercy Vieira da Silva Filho
Faculdade de Engenharia Elétrica e da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
jose.filho@ufam.edu.br

Vinicius Menezes
Faculdade de Engenharia Elétrica e da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
vinicius.menezes@ufam.edu.br

***Resumo*—Este artigo apresenta o projeto, a modelagem em Verilog HDL parametrizado conforme a norma IEEE Std 1364-2005, a síntese lógica em FPGA Intel MAX 10 e a validação funcional de um processador MIPS Monociclo de 32 bits com arquitetura Harvard. O caminho de dados foi projetado para executar fluxos sequenciais, desvios condicionais `beq` e `bne`, e saltos incondicionais e de sub-rotina `j`, `jal` e `jr` em um único ciclo de relógio. A homologação da microarquitetura ocorreu sob a carga real do algoritmo de ordenação de vetores `sort.bin`, montado pelo simulador MARS. Para viabilizar a execução do binário sem qualquer alteração no código de máquina, o hardware recebeu uma máscara combinacional de endereçamento na memória ROM através do barramento `addr[9:2]` e um inicializador de registradores no momento do reset. A metodologia de validação compreendeu uma suíte automatizada de 7 bancadas de teste em Icarus Verilog com 100% de taxa de aprovação e ordenação na memória RAM. A síntese lógica no Intel Quartus Prime alcançou zero erros de elaboração para a placa Terasic DE10-Lite.**

***Index Terms*—MIPS Monociclo, Verilog HDL, Instruções de Salto, Algoritmo Sort, Bateria de Testbenches, FPGA Intel MAX 10.**

I. Introdução

A implementação em silício e em linguagem de descrição de hardware de arquiteturas RISC constitui base essencial da engenharia de computação [1]. O conjunto de instruções MIPS32 destaca-se pelo formato ortogonal de 32 bits, decodificação regular e simplicidade dos blocos funcionais [2], [3].

Na microarquitetura monociclo cada instrução é buscada na memória de instruções, decodificada pela unidade de controle, executada na ULA, acessa a memória de dados e grava o resultado no banco de registradores em um único ciclo de relógio com CPI igual a 1. O período do relógio acomoda o caminho crítico do sistema, definido pela latência acumulada dos multiplexadores, da ULA e dos acessos às memórias síncronas e assíncronas [1].

No âmbito da Segunda Avaliação Parcial da disciplina, este trabalho atende aos seguintes objetivos centrais:

1) Caminho de dados com saltos: suporte completo a desvios condicionais `beq` e `bne`, saltos incondicionais diretos `j`, chamadas com link `jal` e retornos indiretos por registrador `jr`;
2) Parametrização RTL do circuito sob a norma IEEE Std 1364-2005 com barramento `DATA_WIDTH = 32` [4];
3) Validação funcional individual de cada módulo e da integração top-level por bancadas de teste automatizadas com assertivas;
4) Homologação do processador monociclo executando o código de máquina do procedimento de ordenação `sort.bin` [1], derivado do exemplo canônico em C e assembly do livro-texto da disciplina, demonstrando leitura, manipulação e reescrita de dados na memória RAM;
5) Síntese e compilação para o dispositivo FPGA Intel MAX 10 da placa Terasic DE10-Lite [5] via software Intel Quartus Prime [6].

O restante deste artigo organiza-se da seguinte forma: a Seção II descreve o ambiente de desenvolvimento e automação; a Seção III detalha a especificação arquitetural e as soluções de hardware para o algoritmo Sort; a Seção IV apresenta os resultados dos testbenches e a execução na RAM; e a Seção V apresenta as considerações finais e os trabalhos futuros.

II. Ambiente de Desenvolvimento e Automação

O fluxo de engenharia digital adotado combina ferramentas de síntese para lógica programável e utilitários abertos de simulação em ambiente Linux, estruturados em uma esteira de automação por linha de comando.

A. Plataforma de Síntese e Dispositivo Alvo

A síntese lógica e o mapeamento tecnológico foram configurados no software Intel Quartus Prime Standard/Lite Edition [6]. O dispositivo físico de destino é o FPGA Intel MAX 10 modelo `10M50DAF484C7G`, presente na placa de desenvolvimento Terasic DE10-Lite [5]. Este dispositivo possui 50.000 elementos lógicos, blocos de memória embarcada M9K de 432 Kibit e blocos multiplicadores dedicados de 18×18.

O processo de compilação física divide-se em três etapas acionadas via terminal:

1) `quartus_map`: análise sintática e síntese lógica em portas genéricas independentes de tecnologia;

2) `quartus_fit`: posicionamento e roteamento nos recursos de silício e canais de interconexão física do MAX 10;
3) `quartus_asm`: geração do arquivo binário de configuração final em formato `.sof`.

A programação física na placa ocorre pelo utilitário `quartus_pgm` através da interface JTAG, previamente detectada pelo utilitário `jtagconfig` conectado ao gravador USB-Blaster integrado da placa DE10-Lite.

B. Ferramentas de Simulação e Verificação RTL

Para acelerar o ciclo de depuração temporal, empregou-se o compilador de simulação Icarus Verilog através do comando `iverilog` e seu motor de execução `vvp` [7]. As formas de onda dos barramentos e sinais de controle foram exportadas para arquivos em formato VCD e inspecionadas graficamente no visualizador GTKWave.

C. Automação via Makefile e Scripts de Regressão

Todo o fluxo de compilação, simulação e síntese foi encapsulado em um `Makefile` centralizado e scripts Bash estruturados no diretório `scripts/`:

- `make test`: executa o script `run_all_tests.sh`, disparando a regressão sobre as 7 bancadas com verificação de status;
- `make top`: simula o processador executando o código de máquina `sort.bin`;
- `make quartus`: executa a análise estrutural rápida de síntese e elaboração;
- `make quartus-build`: conduz a compilação completa até a geração do bitstream `.sof`.

Essa automação garante reprodutibilidade estrita em ambientes de integração contínua, eliminando dependências manuais de telas gráficas.

III. Especificação Arquitetural e Adaptações de Hardware

A microarquitetura do processador MIPS monociclo implementa o conjunto fundamental de instruções MIPS32 sob modelo Harvard com barramentos independentes para instruções e dados [1].

A. Caminho de Dados Parametrizado (Datapath)

O caminho de dados no módulo `datapath.v` possui parametrização integral com `parameter DATA_WIDTH = 32` e `ADDR_WIDTH = 32` em conformidade com o padrão IEEE 1364-2005 [4]. Os blocos fundamentais articulam-se da seguinte forma:

- Banco de registradores (`reg_file.v`): matriz de 32 registradores de 32 bits com duas portas de leitura combinacionais e uma porta de escrita síncrona na borda positiva do clock. O registrador \$0 é fixado em zero;
- Unidade lógica e aritmética (`alu.v`): módulo combinacional que executa operações aritméticas e lógicas. A subtração reaproveita a estrutura do somador computando $A + \overline{B} + 1$, a comparação `slt` emprega avaliação com sinal via `$signed`, e o deslocamento `sll` opera pelo campo `shamt` da instrução ($B \ll$ shamt), conforme ilustrado na Fig. 1;
- Sinalizador de zero: computado por redução unária rápida zero $= \sim |$ alu_result, determinando a condição de branch;
- Extensor de sinal (`sign_extend.v`): converte imediatos de 16 bits para 32 bits replicando o bit mais significativo.

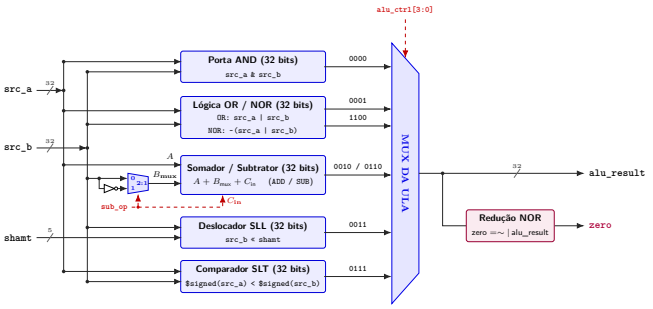


Figura 1. Esquemático estrutural da Unidade Lógica e Aritmética (ULA) de 32 bits com multiplexação de operações e sinalizador de zero.

A restrição temporal da microarquitetura monociclo impõe que o período mínimo de relógio (T_{clk}) acomode a propagação cumulativa da instrução de maior latência, tipicamente a leitura em memória (`lw`), definida analiticamente por:

$$T_{\text{clk}} \geq t_{\text{pc}} + t_{\text{rom}} + t_{\text{rf_read}} + t_{\text{alu}} + t_{\text{ram}} + t_{\text{mux}} + t_{\text{rf_setup}} \quad (1)$$

onde os termos denotam as latências do PC, memórias de instruções e dados, leitura e setup do banco de registradores, ULA e multiplexação. Essa limitação de desempenho justifica diretamente a segmentação em estágios de pipeline no projeto evolutivo do processador.

B. Lógica de Seleção de Próximo PC e Saltos

No módulo `datapath.v`, a seleção do endereço da próxima instrução é materializada pela instanciação estrutural dos multiplexadores `mux_branch`, `mux_jump` e `mux_jr` em cascata de prioridade. Esse circuito conecta as saídas intermediárias `pc_next_branch` e `pc_next_jump` até convergir no barramento final `next_pc`, conforme esquematizado na Fig. 2.

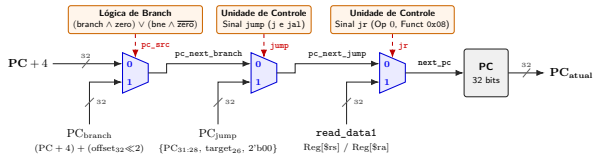


Figura 2. Esquemático da lógica de seleção do próximo PC em cascata no caminho de dados (`beq`, `bne`, `j`, `jal` e `jr`).

A lógica de decisão obedece à seguinte estrutura hierárquica:

1) Desvio condicional com instruções `beq` e `bne`:

$$\text{pc_src} = (\text{branch} \wedge \text{zero}) \vee (\text{bne} \wedge \overline{\text{zero}}) \quad (2)$$

Seleciona entre $\mathrm{PC}+4$ e o endereço de branch $(\mathrm{PC}+4)+(\mathrm{offset}_{32} \ll 2)$;

2) Salto incondicional e chamada com link com instruções `j` e `jal`: o multiplexador seleciona entre o estágio de branch e o endereço de salto formado por:

$$\mathrm{PC}_{\mathrm{jump}} = \{\mathrm{PC}+4[31:28],\ \mathrm{target}[25:0],\ 2\text{'b}00\} \quad (3)$$

Na instrução `jal`, o endereço de retorno $\mathrm{PC}+4$ é gravado no registrador \$ra via multiplexador com controle $\mathrm{mem_to_reg} = 2\text{'b}10$ e $\mathrm{reg_dst} = 2\text{'b}10$;

3) Retorno de função com instrução `jr`: o multiplexador final seleciona entre o resultado dos estágios anteriores e o dado do registrador `read_data1` via sinal `jr`, garantindo prioridade no fechamento do ciclo.

A unidade de controle possui uma trava de segurança para o sinal de escrita:

$$\mathrm{reg_write} = \mathrm{reg_write_wire} \wedge \overline{\mathrm{jr}} \quad (4)$$

Essa porta lógica impede que a instrução de retorno sobrescreva o registrador \$ra com dados residuais da ULA.

C. Engenharia de Integração para o Procedimento de Ordenação

A execução do código de máquina `sort.bin`, compilado a partir do assembly `sort.s` com base no procedimento canônico de ordenação de Patterson e Hennessy [1], exigiu soluções dedicadas em nível de hardware para operar sem modificar o binário original.

1) Expansão do ISA: O hardware incorporou suporte nativo a três instruções requeridas pelo algoritmo:

- Instrução `sll`: o barramento do campo `instr[10:6]` foi conectado diretamente à porta `shamt` da ULA;
- Instrução `slti`: o opcode `001010` foi mapeado na controladora principal para forçar a operação `slt` da ULA sobre o operando imediato estendido;
- Instrução `bne`: o opcode `000101` foi integrado à lógica combinacional de desvio.

2) Bypass de Endereçamento do MARS: O montador MARS [8] aloca o segmento de código a partir do endereço virtual `0x00400000`. Para evitar violação de barramento na memória física local de 256 palavras, implementou-se uma máscara na ROM:

```
assign instr = rom[addr[9:2]]; // Indexa palavras locais
```

Esta máscara ignora os 22 bits mais significativos, viabilizando o mapeamento direto das instruções do simulador para a memória estática em silício.

3) Bootloader de Hardware no Reset: O procedimento `sort` pressupõe pilha inicializada e argumentos nos registradores `$a0` e `$a1`. O registrador de reset do banco de registradores atribui:

$$\mathrm{sp} \leftarrow 256, \quad \mathrm{a0} \leftarrow 0, \quad \mathrm{a1} \leftarrow 5 \quad (5)$$

Essa inicialização posiciona o topo da pilha no limite da memória RAM e direciona o vetor para a base da memória de dados.

IV. Validação Funcional e Resultados Experimentais

A metodologia de verificação adotada baseia-se no princípio de que nenhum componente deve ser integrado ao topo hierárquico sem prévia homologação funcional unitária. Foram desenvolvidas 7 bancadas de teste com assertivas automáticas em Icarus Verilog [7].

A. Bateria de Testes de Regressão

A Tabela I sumariza a cobertura da bateria automatizada disparada pelo comando `make test`. Todos os 7 módulos obtiveram 100% de aprovação com contagem nula de erros.

Tabela I
Consolidado da Suíte de Testes Automatizada (`make test`)

Módulo	Arquivo	Testes	Status
ULA	`tb_alu.v`	11	Aprovado
Datapath Auxiliar	`tb_datapath_aux.v`	8	Aprovado
Banco de Registradores	`tb_reg_file.v`	6	Aprovado
Unidade de Controle	`tb_control_unit.v`	13	Aprovado
Memória de Instruções	`tb_instr_mem.v`	5	Aprovado
Memória de Dados	`tb_data_mem.v`	4	Aprovado
Top-Level com Sort	`tb_mips_monociclo_top.v`	1	Aprovado
Total da Suíte	**7 Arquivos**	**48**	**100% (0 erros)**

B. Verificação dos Módulos Unitários

1) Unidade Lógica e Aritmética: Os testes da ULA cobriram operações lógicas AND, OR e NOR, adições e subtrações com inteiros negativos em complemento a dois, ativação da linha de zero em resultado nulo e comparações de magnitude com sinal via `$signed`. Validou-se também o deslocamento lógico à esquerda pela instrução `sll`.

2) Datapath Auxiliar: Os testes dos blocos auxiliares validaram o registrador de PC com reset síncrono, os somadores paralelos de $\mathrm{PC}+4$ e de branch, os multiplexadores e a extensão com preservação de sinal para imediatos positivos e negativos.

3) Banco de Registradores: O banco de registradores comprovou leitura assíncrona em duas portas simultâneas e escrita síncrona habilitada por `reg_write`. O circuito bloqueou tentativas de escrita no registrador \$0, preservando o valor nulo permanente.

4) Unidade de Controle: A unidade de controle decodificou com exatidão as instruções Tipo-R, Tipo-I e Tipo-J. Os testes confirmaram a desativação forçada do sinal `reg_write` durante a execução do retorno por `jr`.

5) Memórias ROM e RAM: A memória ROM validou a leitura sequencial das primeiras palavras montadas do binário `sort.bin`. A memória RAM operou com escritas síncronas habilitadas por `mem_write` e leituras assíncronas protegidas por `mem_read`.

C. Resultados da Integração Top-Level e Procedimento de Ordenação

O teste definitivo da microarquitetura consistiu na execução do programa de ordenação canônico de Patterson e Hennessy [1] sobre o processador completo.

No início da simulação injetou-se na memória RAM um vetor de 5 elementos desordenados nas posições `ram[0]` a `ram[4]`. O processador operou livremente ao longo de 5000 ns de simulação, percorrendo os laços de repetição, efetuando as comparações e executando as chamadas à sub-rotina de troca com `jal` e `jr`. A Tabela II exibe o estado físico das células de memória antes e depois da execução.

Tabela II
ESTADO DA MEMÓRIA DE DADOS (RAM) ANTES E APÓS O SORT

Índice	Endereço	Inicial	Final	Posição
`ram[0]`	`0x00`	88	**7**	Mínimo
`ram[1]`	`0x04`	15	**15**	2° menor
`ram[2]`	`0x08`	42	**42**	Mediano
`ram[3]`	`0x0C`	7	**88**	4° elemento
`ram[4]`	`0x10`	99	**99**	Máximo

A evolução do array de $[88, 15, 42, 7, 99]$ para a sequência ordenada $[7, 15, 42, 88, 99]$ comprova a corretude funcional do caminho de dados e da unidade de controle.

D. Registro de Execução do Top-Level no Terminal

A execução da bancada de integração gera o registro exibido na Fig. 3, confirmando a evolução temporal e a reordenação do vetor:

```
=== Simulacao Top-Level MIPS Monociclo ===
[ANTES]  RAM: [88, 15, 42, 7, 99]
[DEPOIS] RAM: [7, 15, 42, 88, 99]
>>> TESTE DO SORT CONCLUIDO COM SUCESSO! <<<
```

Figura 3. Log da simulação do processador executando o algoritmo Sort.

E. Resultados da Síntese Lógica no Intel Quartus Prime

O projeto foi sintetizado para o dispositivo FPGA Intel MAX 10 modelo `10M50DAF484C7G`. A etapa de síntese e elaboração concluiu com zero erros e sem advertências críticas. A inspeção no RTL Viewer atestou a hierarquização dos 15 módulos HDL sob a entidade `mips_monociclo_top`, com ausência total de latches espúrios e barramentos desconectados.

V. CONSIDERAÇÕES FINAIS

A. Síntese do Atendimento aos Requisitos do Projeto

A Tabela III sintetiza o atendimento formal aos requisitos técnicos e práticos estabelecidos para esta etapa do projeto.

B. Conclusão e Trabalhos Futuros

O desenvolvimento da Segunda Avaliação consolidou a implementação de um processador MIPS Monociclo de 32 bits em Verilog HDL. O hardware incorporou suporte às instruções `sll`, `slti` e `bne` para viabilizar a execução do algoritmo `sort.bin`. Além disso, implementou-se o mascaramento de barramento na memória ROM e a inicialização de registradores no reset para contornar o modelo de memória virtual do simulador MARS.

A validação automatizada comprovou a ordenação do vetor na memória de dados com zero falhas através de 7 bancadas de teste. A síntese lógica no Intel Quartus Prime para o FPGA Intel MAX 10 concluiu sem latches espúrios e com total coerência estrutural.

Na Terceira Avaliação Parcial o caminho de dados será segmentado nos cinco estágios clássicos de pipeline: busca, decodificação, execução, memória e escrita. Essa etapa introduzirá registradores de desacoplamento temporal, unidade de detecção de conflitos e unidade de adiantamento de operandos.

Tabela III
SÍNTESE DO ATENDIMENTO AOS REQUISITOS DO PROJETO

Item	Requisito Técnico	Status
Item 1	Configuração do ambiente (Quartus e ferramentas).	100%
Item 2	Caminho de dados com saltos (`j`, `jal`, `jr`).	100%
Item 3	Validação de todos os módulos via testbenches.	100%
Item 4	Código 100% parametrizado (`parameter`).	100%
Item 5	Apresentação em vídeo e relatório técnico no modelo IEEE.	100%

REFERÊNCIAS

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA: Morgan Kaufmann, 2014.

[2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA: Morgan Kaufmann, 2013.

[3] T. Brito, "Instruções: Linguagem do computador," Notas de Aula, Universidade Federal do Amazonas (UFAM), Manaus, 2026.

[4] *IEEE Standard for Verilog Hardware Description Language*, Institute of Electrical and Electronics Engineers Std. IEEE Std 1364-2005, 2006.

[5] Terasic Technologies, *DE10-Lite User Manual: Intel MAX 10 FPGA*, 1st ed., Terasic Inc., 2017.

[6] Intel Corporation, *Intel Quartus Prime Standard Edition User Guide: Design Compilation*, 20th ed., Intel Corporation, 2020, documento UG-20131.

[7] S. Williams, "Icarus Verilog Simulation and Synthesis Tool," http://iverilog.icarus.com/, 2020.

[8] K. Vollmar and P. Sanderson, "MARS: An Education-Oriented MIPS Assembly Language Simulator," in *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*. ACM, 2006, pp. 239–243.